

Die Vitalzeichen einer Plattform: Wie man mit den Four Golden Signals den Gesundheitszustand von frag.jetzt überwacht

Exposé zur Bachelorarbeit im Studiengang Informatik

Max Mustermann

27. Januar 2026

Zusammenfassung

Moderne Webplattformen wirken nach außen oft stabil und reaktionsschnell, doch ihr innerer Zustand bleibt im laufenden Betrieb häufig verborgen. Dieser Artikel zeigt, wie Observability dabei hilft, verteilte Systeme wie die Open-Source-Plattform *frag.jetzt* als Ganzes zu verstehen und ihren „Gesundheitszustand“ sichtbar zu machen. Anhand der Four Golden Signals wird diskutiert, welche technischen Anzeichen tatsächlich aussagekräftig sind, warum isoliertes Monitoring nicht ausreicht und wie Metriken, Logs, Traces und intelligentes Alerting zusammenspielen müssen, um handlungsrelevante Einblicke zu liefern. Im Fokus steht dabei weniger die Theorie einzelner Werkzeuge als vielmehr die Frage, wie aus Rohdaten Entscheidungen werden, die Betrieb, Entwicklung und Produktverantwortung gleichermaßen unterstützen.

und komplexe Berechnungen landen schließlich in einem Python-basierten KI-Backend, das wiederum externe LLM-APIs einbindet [1]. Was früher ein einzelner Fehler war, ist nun häufig eine Kettenreaktion.

Wenn hier etwas hakt, ist das System selten komplett „tot“ — es ist „krank“. Es leidet an unsichtbaren Entzündungen: Ein einzelner Service antwortet *beispielhaft* 200 Millisekunden zu spät, und aus dieser Verzögerung wird in der Summe ein Stillstand, der beim Nutzer als totale Blockade ankommt [2].

Das Problem: Herkömmliches Monitoring sagt uns oft nur: „Der Server ist an.“ Es sagt nicht, *warum* sich die Anwendung für den Nutzer zäh anfühlt. Hier wechseln wir die Strategie — von reiner Überwachung zu **Observability**. Wir wollen nicht nur wissen, ob das Herz noch schlägt, sondern ob der Patient wirklich stabil ist. Dafür nutzen wir die *Four Golden Signals* als medizinische Vitalwerte [3].

1 Notaufnahme für den Code: Warum wir den Puls von frag.jetzt sichtbar machen müssen

Stellen Sie sich folgende Szene vor: Ein vollbesetzter Hörsaal, 500 Studierende starren auf die Leinwand. Der Dozent hat gerade eine interaktive Frage über *frag.jetzt* gestartet. 500 Daumen tippen auf Smartphones. Absenden.

Und dann: **Stille.**

Auf der Leinwand dreht sich der Ladebalken. Sekunden vergehen, die sich wie Minuten anfühlen. Ist das WLAN der Uni zusammengebrochen? Hat sich der Datenbank-Pool erschöpft? Oder wartet das Backend gerade vergeblich auf eine Antwort der externen KI-Schnittstelle?

In der „guten alten Zeit“ monolithischer Software hätte man in eine Log-Datei geschaut und die Antwort gefunden. Doch *frag.jetzt* ist heute ein cloud-natives Nervensystem: Ein Angular-Frontend spricht mit einem NGINX-WebSocket-Gateway, das Anfragen an ein Spring-Boot-Backend weiterreicht; dort wird zur Identitätsprüfung ein externer Dienst (Keycloak) kontaktiert, Nachrichten werden über RabbitMQ verteilt,

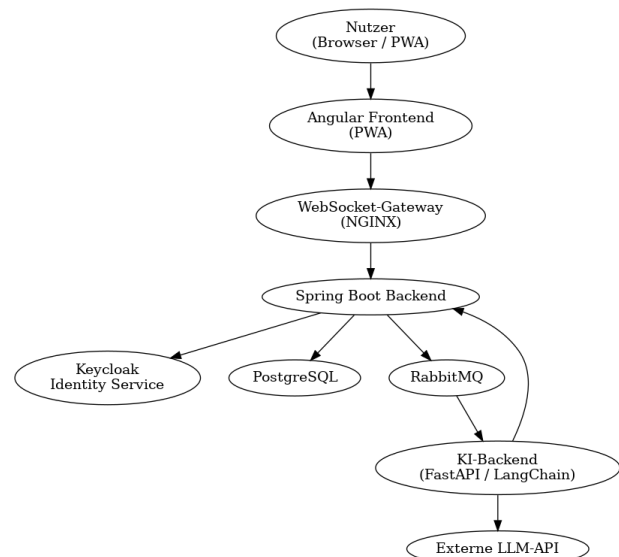


Abbildung 1: Das „Nervensystem“ des Patienten: Der vereinfachte Anfragepfad bei *frag.jetzt*. Eigene Darstellung nach [1].

Dieser Artikel zeigt, wie *frag.jetzt* in die Diagnose-Röhre geschoben wird, damit aus stummen Daten sprechende Antworten werden — und warum „Ich

glaube, es hängt“ keine gültige Fehlerbeschreibung mehr ist.

2 Vier Golden Signals: Der Gesundheits-Check statt Symptom-Raten

Wenn ein Patient in die Notaufnahme kommt, misst der Arzt nicht „alles“. Er schaut auf wenige Vitalwerte, die schnell Auskunft geben. In der Welt der Google Site Reliability Engineers (SRE) haben sich dafür vier universelle Messkategorien etabliert, die sogenannten *Four Golden Signals* [3]. Für *frag.jetzt* markieren sie die Grenze zwischen „Ich hoffe, es läuft“ und „Ich weiß, was los ist“.

2.1 Latenz: Die Zeit zwischen Klick und Erlösung

Es ist egal, dass die Datenbank eine Anfrage intern in *beispielhaften* 5 Millisekunden verarbeitet, wenn der Nutzer im Hörsaal über schwaches WLAN erst nach drei Sekunden eine Antwort sieht. Latenz ist die Zeit, die eine Anfrage insgesamt braucht, um bedient zu werden.

Der Trugschluss: Der Durchschnitt beruhigt. Wenn 90 % der Anfragen schnell sind, aber die langsamsten 10 % (die Dozenten-Fragen, die gerade alle sehen) hängen, haben wir ein Problem. Deshalb wird nicht nur der Mittelwert betrachtet, sondern insbesondere die *Tail Latency* — also wie lange die langsamsten Anfragen dauern [4]. Genau dort entsteht Frust, und genau dort verstecken sich die harten Fälle.

2.2 Traffic: Der Herzschlag des Hörsaals

Wie viel Last trifft unser System gerade? Bei *frag.jetzt* ist das kein stetiges Rauschen, sondern manchmal ein Tsunami.

Das Szenario: Die Vorlesung beginnt. 500 Studierende öffnen gleichzeitig die App. Der Traffic schießt *beispielhaft* von 0 auf 500 Anfragen pro Sekunde. Ein plötzlicher Abfall auf Null ist hier übrigens genauso verdächtig wie eine Überlastung: Er bedeutet oft nicht „Feierabend“, sondern „Netzwerkfehler“ oder „Gateway hängt“ [3].

2.3 Fehler: Wenn das Schweigen laut wird

Klassische Fehler sind einfach: Ein HTTP-Code 500 (Internal Server Error) ist ein Schrei nach Hilfe. Doch in verteilten Systemen lauern die gefährlicheren Varianten im Flüsterton.

Die Tücke: Viel bedrohlicher sind die „leisen“ Fehler. Wenn das Backend zwar „Code 200 OK“ antwortet, aber der Inhalt leer ist oder falsche Daten liefert, merkt das klassische Monitoring nichts — der Nutzer aber alles. Observability bedeutet hier, tief genug in

die Antwort zu schauen, um auch funktionale Pannen zu erkennen [3]. Das kann bedeuten, dass semantische Checks (z. B. „Antwort enthält mindestens X Elemente“) instrumentiert und als Fehlerrate sichtbar gemacht werden.

2.4 Sättigung: Der Stau vor der Kasse

Sättigung misst, wie „voll“ Ressourcen sind: CPU, Speicher, Thread-Pools, Datenbank-Connection-Pools. Sie ist oft der wichtigste Indikator für das, was als Nächstes passiert [3].

Die Analogie: Stellen Sie sich die Warteschlange in der Mensa vor. Solange die Schlange sich bewegt, ist alles im Rahmen. Aber wenn die Mensa zu 99 % gefüllt ist, reicht eine einzige weitere Person, und alles kommt zum Stillstand. Sättigung warnt *bevor* das System langsam wird — sie ist ein Frühwarnsystem [3].

2.5 Fazit der Diagnose: Vier Werte, eine Geschichte

Ein einzelner Messwert ist fast immer missverständlich. Hohe Latenz kann an hoher Sättigung liegen — oder an einem Netzwerkproblem. Erst wenn wir diese vier Vitalwerte übereinanderlegen (Korrelation), wird aus Daten echtes Wissen [3]. Und erst dann wird Debugging weniger Ratespiel und mehr Detektivarbeit mit Beweismitteln.

Signal	frag.jetzt: Messpunkt & Alarm-Beispiel
Latenz	p95/p99 Antwortzeit für „Frage senden“ im Gateway/Backend; Alarm bei anhaltender p99-Erhöhung (Symptom, nicht Ursache) [3, 4].
Traffic	Requests/s (HTTP/WebSocket), Queue-Throughput; Alarm bei abruptem Abfall auf 0 oder unplausiblen Peaks [3].
Fehler	5xx, Timeouts, semantische Checks (z. B. leere/inkonsistente Antwort); Alarm bei signifikanter Fehlerquote [3].
Sättigung	CPU/RAM, Connection-Pool, Queue-Lag; Alarm bei dauerhaftem Annähern an Limits bzw. wachsendem Lag [3].

Tabelle 1: Four Golden Signals als „Vitalwerte“: kompaktes Mapping auf *frag.jetzt*.

3 Werkzeug-Tetris: Die Suche nach dem passenden Stack

Wer heute „Observability Tools“ in eine Suchmaschine eingibt, wird von einer Lawine begraben. Es gibt hunderte Dienste, vom teuren Enterprise-SaaS bis zum selbstgebaute Skript. Für ein Open-Source-Projekt wie *frag.jetzt* ist klar: Wir können uns keine Lizenzkosten leisten, aber wir können uns auch keine blinden Flecken leisten [5].

Wir standen vor der klassischen Frage: Was brauchen wir wirklich — einen Detektiv, einen Wächter oder ein Navigationssystem? Die Antwort lautet: alles drei, aber in der richtigen Dosierung [5].

3.1 Der Wächter: Prometheus für den Puls

Unsere erste Wahl fiel auf **Prometheus**. Warum? Weil Logs teuer sind: Jede Zeile Text zu speichern, kostet Speicherplatz und Rechenleistung. Metriken hingegen sind kompakt, weil sie Zahlenreihen darstellen. Prometheus speichert keine Romane, sondern Messwerte wie „CPU bei 80 %“ oder „50 Anfragen pro Sekunde“ [6]. Das ist wie ein EKG: Es zeigt Stress, aber nicht zwangsläufig die Ursache.

Für *frag.jetzt* bedeutet das: Der Puls wird überall fühlbar gemacht — an den Schnittstellen, an kritischen Pools, an den Endpunkten, die Nutzer wirklich spüren.

3.2 Der Detektiv: Logs (aber bitte gezielt)

Wenn Prometheus Alarm schlägt, braucht das Team Details. Hier kommt klassisch der **ELK-Stack** (Elasticsearch, Logstash, Kibana) ins Spiel. Aber Vorsicht: In verteilten Systemen explodieren Logs förmlich; „alles loggen“ ist selten nachhaltig [5].

Wir haben gelernt: „Log everything“ ist keine Strategie, sondern ein Weg, die Festplatte zu verstopfen.

Sinnvoller ist es, Logs dort zu setzen, wo Entscheidungen getroffen werden: bei Fehlermeldungen, bei Zustandswechseln, beim Start der Anwendung — nicht bei jedem Herzschlag [7]. So wird der Detektiv nicht zum Messie.

3.3 Das Navi: Distributed Tracing mit Jaeger

Hier wurde es für *frag.jetzt* kritisch. Metriken und Logs sind mächtig, aber ohne Kontext oft isoliert. Sie sagen: „Die Datenbank ist langsam“ und „Das Frontend wartet“. Aber sie beantworten nicht: *Worauf* wartet das Frontend eigentlich, und *welcher* Call hat den Stau ausgelöst?

In einer Microservice-Architektur wie unserer verliert man ohne **Tracing** den Faden [8]. Deshalb wurde Jaeger eingesetzt, um den Weg einer einzelnen Anfrage über Gateway, Spring Boot und KI-Backend nachzuzeichnen. Erst Tracing macht aus einzelnen Bauteilen eine zusammenhängende Geschichte [8].

3.4 Die Geheimwaffe: OpenTelemetry gegen blinde Flecken

Das größte Risiko bei der Tool-Auswahl ist der „Vendor Lock-in“. Was, wenn morgen von Jaeger zu Tempo gewechselt wird? Oder von Prometheus zu einer Cloud-Lösung? Eine Observability-Strategie sollte nicht auf Werkzeug-Markennamen beruhen, sondern auf austauschbaren Standards.

Hier setzt **OpenTelemetry (OTel)** an: als universeller Adapter, der Instrumentierung vom Backend bis zum Gateway neutral hält und Daten an die Tools der Wahl verteilt [8]. Statt Code mit einer Prometheus-spezifischen Bibliothek zu „verdrahten“, wird alles über OTel gesammelt und exportiert. Das reduziert Abhängigkeiten und erleichtert spätere Entscheidungen [8].

Rolle	Baustein & Fokus
Wächter	Prometheus + Grafana: Trends, Ausreißer, Ressourcendruck; Alarme auf Nutzer-Symptome [6, 3].
Detektiv	ELK/Logs: Ursachenanalyse, Zustandswechsel, Fehlerkontext; Logging sparsam und strukturiert [5, 7].
Navi	Jaeger/Tracing: End-to-End-Requestpfad; Engpässe lokalisieren, Korrelation sichern [8].
Klebstoff	OpenTelemetry: einheitliche Instrumentierung, weniger Lock-in, konsistente Attribute [8].

Tabelle 2: Observability-Stack als Team: Rollen statt Tool-Fetisch.

4 Schluss mit dem digitalen Wecker: Warum Stille Gold ist

Hand aufs Herz: Nichts ruiniert die Moral eines Teams schneller als ein Telefon, das nachts um 3 Uhr vibriert, nur um zu melden: „CPU-Auslastung bei 85 %“ — und fünf Minuten später springt der Wert von selbst wieder auf „OK“.

Dieses Phänomen hat einen Namen: **Alarmmüdigkeit** (Alert Fatigue) [9]. Wenn alles ein Notfall ist, ist nichts mehr ein Notfall. In der frühen Phase von *frag.jetzt* neigten wir dazu, auf alles zu lauschen: Ein Service antwortet nicht sofort? Alarm. Die Datenbank schreibt viele Logs? Alarm. Das Ergebnis: Benachrichtigungen wurden ignoriert, wie man das Rauschen einer Klimaanlage irgendwann ausblendet. Und genau dann passierte der echte Ausfall.

Also wurde das Alarm-Konzept radikal umgestellt — weg von „Rauschen“, hin zu „Handlung“. Drei Regeln haben sich dabei als Leitplanken bewährt, weil sie Alarme zu Entscheidungen machen und nicht zu Nervenkrieg [3] [9]:

1. **Weck mich nur, wenn der Nutzer leidet.** Ein hoher CPU-Wert ist kein Grund zur Panik, solange die Antworten schnell kommen. Alarmiert wird auf *Symptome*, nicht auf bloße Ursachen [3]. *Schlecht:* „Server Node 4 hat hohe Last.“ *Gut:* „Die p99-Latenz beim Absenden von Fragen ist (beispielhaft) über 2 Sekunden gestiegen“ [4][3].
2. **Die „Luft-anhalten“-Strategie.** Netzwerke haben Schluckauf. Das ist normal. Statt bei jedem verlorenen Paket sofort „Feuer!“ zu schreien, wird zeitlich geglättet. Besteht das Problem länger als eine definierte Dauer (z. B. zwei Minuten), wird die Nachricht erst dann aktiv. Das eliminiert nervöse Fehlalarme („Flapping“) fast vollständig [3].
3. **Kein Alarm ohne Spielbuch.** Ein Alarm, der nur sagt „Es ist kaputt“, ist wertlos. Er erzeugt Stress, aber keine Lösung. Deshalb werden **Runbooks** direkt mit Alerts verknüpft: Wenn der „Datenbank-Pool erschöpft“-Alarm kommt, liefert er die Anlei-

tung gleich mit: „(1) Offene Verbindungen prüfen, (2) Service X neu starten, (3) Lastspitze korrelieren“ [3][9].

4.1 Ein kurzer Einschub: Warum das mehr ist als Tool-Kosmetik

Ein gutes Observability-System ist wie ein guter Rauchmelder: Es schweigt die meiste Zeit. Aber wenn es sich meldet, dann schnappt man sich den Feuerlöscher — und nicht die Ohropax. Diese Logik wird nicht durch „mehr Daten“ erreicht, sondern durch *bessere* Daten, durch Korrelation und durch Alarme, die Handlung erzwingen statt Ablenkung produzieren [3][9].

5 Fazit: Wenn aus Stille Vertrauen wird

Kehren wir zurück in Hörsaal 1. Wieder zücken 500 Studierende ihre Smartphones. Wieder stellt der Dozent eine Frage über *frag.jetzt*. Und wieder gibt es im Hintergrund vielleicht ein technisches Problem — eine Datenbankabfrage dauert *beispielhaft* 200 Millisekunden zu lang.

Der Unterschied zu früher? Niemand merkt es.

Bevor der Ladebalken auf der Leinwand überhaupt ins Stocken gerät, hat der Observability-Stack das Symptom („Latenz steigt am Backend“) bereits erfasst. Prometheus hat den Trend erkannt, noch bevor ein Grenzwert gerissen wurde [6]. Ein Blick in die Tracing-Ansicht zeigt: „Aha, der Connection-Pool zur Datenbank läuft voll“ [8]. Und das Runbook liefert die Lösung, noch bevor der erste Nutzer frustriert die App schließen kann [3][9].

Das ist der Kern von Observability: Es geht nicht um bunte Dashboards an der Wand, die beeindruckend aussehen. Es geht darum, „Unbekannte Unbekannte“ beherrschbar zu machen [4]. Fehler in verteilten Systemen werden nicht vollständig verhindert werden können — Systeme *werden* ausfallen [3]. Aber wir können entscheiden, wie schnell wir verstehen, was passiert, und wie schnell wir wieder stabil sind.

5.1 Der nächste Schritt: Von „Best Effort“ zu Service Level Objectives (SLOs)

Sind wir fertig? Nein. Observability ist kein Projekt, sondern ein Prozess. Aktuell wird gemessen, was *ist*. Der nächste Schritt ist die Definition dessen, was *sein darf*. Mit **Service Level Objectives (SLOs)** wird festgelegt, welche Nutzererfahrung als „gut genug“ gilt, etwa: „99 % aller Fragen müssen in unter 500 ms beantwortet sein“ [3]. Das zwingt dazu, Technik nicht nur technisch zu betrachten, sondern vom Nutzer her zu denken. Und es schafft eine klare Konsequenz: Wenn das Ziel verfehlt wird, werden neue Features gestoppt und Stabilität priorisiert [3].

Am Ende ist *frag.jetzt* mehr als Code. Es ist die Interaktion zwischen Menschen. Und die Aufgabe einer Observability-Strategie besteht darin, sicherzustellen, dass die Technik dieser Interaktion niemals im Weg steht.

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Exposés wurden KI-Tools (ChatGPT 5.2, Google Gemini Pro 3, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.

Quellen und weiterführende Literatur

- [1] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 20.01.2026).
- [2] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [3] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017.
- [4] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [6] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.
- [7] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
- [8] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.
- [9] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.